ByteMart Digital

THE COMPLETE GUIDE

Java Advanced

From Fundamentals to Professional Practice

A deep, example-driven guide to advanced Java — modern language features, generics, functional design, concurrency, virtual threads, JVM internals, and performance.

30	200+	14	8
Chapters	Code Examples	Visual Diagrams	Modules

© 2026 ByteMart Digital · All Rights Reserved

ByteMart Digital

Java Advanced: The Complete Guide

First Edition · 2026

Copyright © 2026 ByteMart Digital. All rights reserved.
No part of this publication may be reproduced, distributed, or transmitted in any form or by any means — including photocopying, recording, screenshotting, or other electronic or mechanical methods — without the prior written permission of the publisher, except for brief quotations used in reviews and certain other non-commercial uses permitted by copyright law.

This product is licensed for personal use by the purchaser only. Sharing, reselling, or redistributing this file in whole or in part is strictly prohibited and constitutes a violation of copyright law.

Disclaimer
The information in this book is provided in good faith and for educational purposes. While every effort has been made to ensure accuracy, the author and publisher assume no responsibility for errors, omissions, or outcomes arising from the use of the code or information contained herein. All code examples are provided "as is".

Trademarks
Java is a registered trademark of Oracle and/or its affiliates. All other product names, logos, and brands are the property of their respective owners and are used for identification purposes only.

Published by ByteMart Digital

Web: bytemartdigital.com
Email: contact@bytemartdigital.in
Instagram: instagram.com/bytemartdigital.in
Telegram: t.me/bytemartcodingcommunity

Contents

1. Records in Depth 4
2. Sealed Classes & Interfaces 7
3. Pattern Matching 10
4. Text Blocks & Switch Expressions 13
5. Type Erasure & Its Consequences 16
6. Wildcards & PECS 19
7. Generic Methods & Type Inference 22
8. Functional Interfaces in Depth 25
9. Function Composition 28
10. Advanced Streams 31
11. Custom Collectors 34
12. The Java Memory Model 37
13. Locks & Synchronizers 40
14. Concurrent Collections & Atomics 43
15. Thread Coordination 46
16. CompletableFuture 49
17. Executors & Fork/Join 52
18. Parallel Streams 55
19. Virtual Threads 58
20. Structured Concurrency 61
21. Class Loading 64
22. JIT Compilation 67
23. Garbage Collection Deep Dive 70
24. Memory & Off-Heap 73
25. Benchmarking with JMH 75
26. Reflection & Method Handles 78
27. Annotation Processing 81
28. The Module System (JPMS) 84
29. Serialization 87
30. Design Patterns in Modern Java 90

MODULE 1 · MODERN LANGUAGE FEATURES

CHAPTER 01

Records in Depth

Immutable data carriers beyond the basics — compact constructors, validation, and idioms.

Records give you immutable data classes for free, but their real power shows in the details: compact constructors for validation, custom accessors, static factories, and how they pair with sealed types and pattern matching. This chapter goes past the one-liner.

What the compiler generates

A record's **components** become `private final` fields, a canonical constructor, accessor methods, and correct `equals`, `hashCode`, and `toString` — all derived from the component list.

```java
record Range(int low, int high) { }
// generates: low(), high(), equals, hashCode, toString,
// and the canonical constructor Range(int, int)
```

Compact constructors for validation

A **compact constructor** omits the parameter list and runs before the fields are assigned — the natural place to validate or normalise inputs.

```java
record Range(int low, int high) {
    Range {                          // compact constructor
        if (low > high)
            throw new IllegalArgumentException("low > high");
    }
}
```

Custom members

Records can add methods, static factories, and even override an accessor — but they cannot add instance fields, keeping them genuinely immutable.

```java
record Money(long cents) {
    static Money ofRupees(double r) { return new Money((long)(r * 100)); }
    double rupees() { return cents / 100.0; }   // derived, not stored
}
```

Where records shine

- DTOs, value objects, and method results that bundle several values.

- Keys in maps — correct `equals/hashCode` come for free.
- The 'leaves' of a sealed hierarchy, deconstructed by pattern matching.
- Local records inside a method for temporary grouping.

WATCH OUT

Records are shallowly immutable. A record holding a mutable object (like a `List`) can still have that object changed — defensively copy in the compact constructor if you need deep immutability.

Immutable data, done right

Records remove the boilerplate of value classes while giving you validation hooks and perfect equality. Treat them as the default for immutable data and the building blocks of sealed, pattern-matched designs.

© 2026 ByteMart Digital

MODULE 1 · MODERN LANGUAGE FEATURES

CHAPTER 02

Sealed Classes & Interfaces

Restricting which types can extend or implement a type — for safer, exhaustive hierarchies.

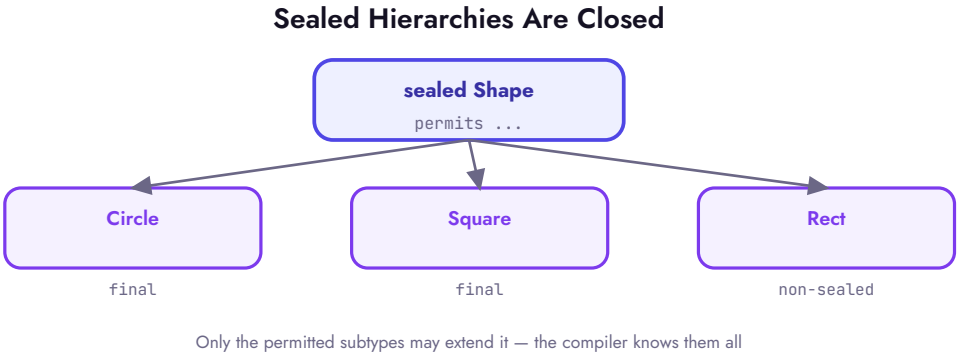
A **sealed** type declares exactly which classes may extend or implement it. This closes a hierarchy so the compiler knows every possible subtype — enabling exhaustive pattern matching and modelling fixed sets of alternatives with full type safety.

> **Sealed type**
> A class or interface that permits only a listed set of direct subtypes, declared with `sealed ... permits`.

Declaring a sealed hierarchy

```java
sealed interface Shape permits Circle, Square, Rectangle { }

record Circle(double radius) implements Shape { }
record Square(double side) implements Shape { }
record Rectangle(double w, double h) implements Shape { }
```



A sealed type permits only its listed subtypes; the compiler knows them all.

The three permitted modifiers

Each permitted subtype must choose how it continues the hierarchy: `final` (closed), `sealed` (further restricted), or `non-sealed` (reopened for extension).

Modifier	Means
final	no further subtypes
sealed	permits its own listed subtypes
non-sealed	open again for any subtype

Exhaustiveness

Because the compiler knows all subtypes, a `switch` over a sealed type can be checked for exhaustiveness — no `default` needed, and adding a new subtype forces you to update every switch.

java

```
double area(Shape s) {
    return switch (s) {                // no default needed
        case Circle c    -> Math.PI * c.radius() * c.radius();
        case Square sq   -> sq.side() * sq.side();
        case Rectangle r -> r.w() * r.h();
    };
}
```

Closed sets, checked by the compiler

Sealed types let you model a fixed set of alternatives and have the compiler guarantee you handle every one. Combined with records and pattern matching, they bring algebraic-data-type safety to Java.

MODULE 1 · MODERN LANGUAGE FEATURES

CHAPTER 03

Pattern Matching

Testing type and shape in one step — instanceof patterns, switch patterns, and record deconstruction.

Pattern matching lets you test an object's type and bind it to a variable in a single step, then deconstruct it into its parts. It replaces long chains of `instanceof` and casting with concise, safe, readable code.

instanceof patterns

The `instanceof` pattern tests the type and, if it matches, binds a variable of that type — no separate cast.

java

```
// Old
if (obj instanceof String) {
    String s = (String) obj;
    System.out.println(s.length());
}
// Pattern matching
if (obj instanceof String s) {
    System.out.println(s.length());   // s already typed
}
```

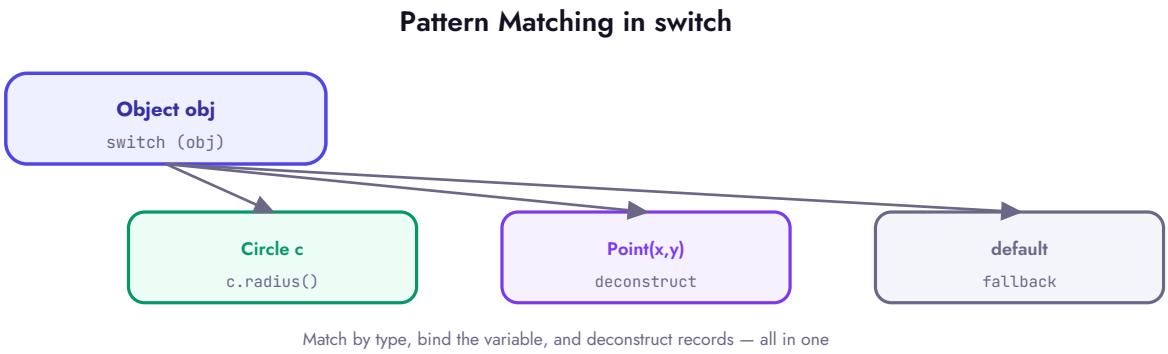
switch patterns

A `switch` can branch on the type of its subject, binding a typed variable per case. Over a sealed type it is exhaustive.

java

```
String describe(Object o) {
    return switch (o) {
        case Integer i when i > 0 -> "positive int";   // guarded
        case Integer i            -> "int";
        case String s             -> "string of " + s.length();
        case null                 -> "null";
        default                   -> "other";
    };
}
```

© 2026 ByteMart Digital



Match by type, bind the variable, guard with when, and deconstruct records.

Record deconstruction

A **record pattern** destructures a record into its components directly in the pattern, nesting to any depth — ideal for sealed hierarchies of records.

java

```
record Point(int x, int y) { }
record Line(Point from, Point to) { }

String show(Object o) {
    return switch (o) {
        case Line(Point(var x1, var y1), Point(var x2, var y2)) ->
            "(%d,%d)->(%d,%d)".formatted(x1, y1, x2, y2);
        default -> "?";
    };
}
```

TIP

Combine sealed types, records, and switch patterns to model and process data the way functional languages do — exhaustive, type-safe, and free of casts.

Test, bind, and destructure

Pattern matching collapses type checks, casts, and extraction into one expression. With record patterns and sealed types it turns branching over data shapes into clean, compiler-checked code.

MODULE 1 · MODERN LANGUAGE FEATURES

CHAPTER 04

Text Blocks & Switch Expressions

Multi-line string literals and value-returning switch — two quiet quality-of-life upgrades.

Two modern features remove long-standing friction: **text blocks** make multi-line strings readable, and the **switch expression** returns a value with no fall-through. Both are small but appear constantly in real code.

Text blocks

A text block is a multi-line string literal delimited by triple quotes. It preserves formatting and strips incidental indentation, making embedded JSON, SQL, and HTML far cleaner.

```java
String json = """
    {
        "name": "ByteMart",
        "active": true
    }
    """;
// No escaped quotes, no concatenation, real line breaks
```

Switch expressions

The arrow-style `switch` returns a value, supports multiple labels per case, and needs no `break` — eliminating the classic fall-through bug.

```java
int days = switch (month) {
    case FEB -> isLeap ? 29 : 28;
    case APR, JUN, SEP, NOV -> 30;
    default -> 31;
};
```

yield for blocks

When a case needs several statements, use a block and `yield` to produce the value.

```java
int size = switch (type) {
    case LARGE -> 100;
    case CUSTOM -> {
        int base = lookup(type);
        yield base * factor;      // return a value from the block
    }
    default -> 0;
};
```

Why they matter

- Text blocks make embedded formats **readable** and diff-friendly.
- Switch expressions are **expressions** — assign or return them directly.
- No fall-through means one less **whole class of bug**.
- Both pair naturally with records and pattern matching.

Small features, everyday wins

Text blocks and switch expressions are not glamorous, but they remove noise from code you write daily — cleaner strings and safer, value-returning branches.

© 2026 ByteMart Digital

MODULE 2 · ADVANCED GENERICS

CHAPTER 05

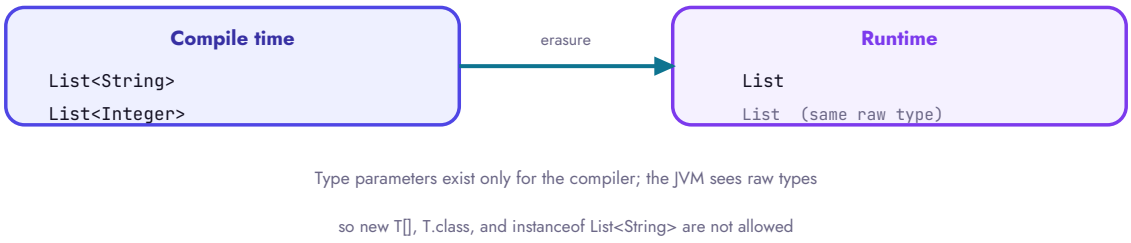
Type Erasure & Its Consequences

How generics vanish at runtime, and the surprising limitations that follow from it.

Java generics exist only at compile time. The compiler checks types, then **erases** them — `List<String>` and `List<Integer>` both become a raw `List` at runtime. Understanding erasure explains a whole set of otherwise-baffling restrictions.

> **Type erasure**
> The process by which the compiler removes generic type information after checking it, replacing type parameters with their bounds (or Object) in the bytecode.

Generics Are Erased at Runtime



Type parameters exist only for the compiler; the JVM sees raw types

so new T[], T.class, and instanceof List<String> are not allowed

Type parameters exist only for the compiler; the JVM sees the raw type.

What erasure forbids

- No `new T[]` — the array type is unknown at runtime.
- No `T.class` or `new T()` — the type is gone.
- No `obj instanceof List<String>` — only the raw type is checkable.
- Two methods differing only by generic parameter clash after erasure.

The bridge and the cast

Because types are erased, the compiler inserts hidden casts and 'bridge methods' so overriding and generic calls still work. You rarely see them, but they are why generics stay backward compatible with pre-generics code.

Working around it with Class tokens

When you genuinely need the runtime type, pass a `Class<T>` token explicitly — the standard workaround for erasure.

© 2026 ByteMart Digital

```java
static <T> T parse(String s, Class<T> type) {
    return type.cast(convert(s, type));   // type known via the token
}
Integer n = parse("42", Integer.class);
```

Heap pollution and @SafeVarargs

Generic varargs create arrays of an erased type, risking **heap pollution**. When your method is genuinely safe, annotate it `@SafeVarargs` to suppress the warning.

WATCH OUT

Never mix raw types with generics to 'get around' a compile error. It silences the compiler's safety checks and reintroduces the runtime `ClassCastException`s generics exist to prevent.

Compile-time only

Generics are a compile-time contract that disappears at runtime. Nearly every generics restriction — no `new T[]`, no reified checks — traces back to erasure. When you need runtime types, use a `Class` token.

MODULE 2 · ADVANCED GENERICS

CHAPTER 06

Wildcards & PECS

Bounded wildcards and the Producer-Extends, Consumer-Super rule for flexible generic APIs.

Wildcards make generic APIs flexible without sacrificing safety. The trick is knowing when to use `? extends` versus `? super` — captured by the famous mnemonic **PECS: Producer Extends, Consumer Super**.

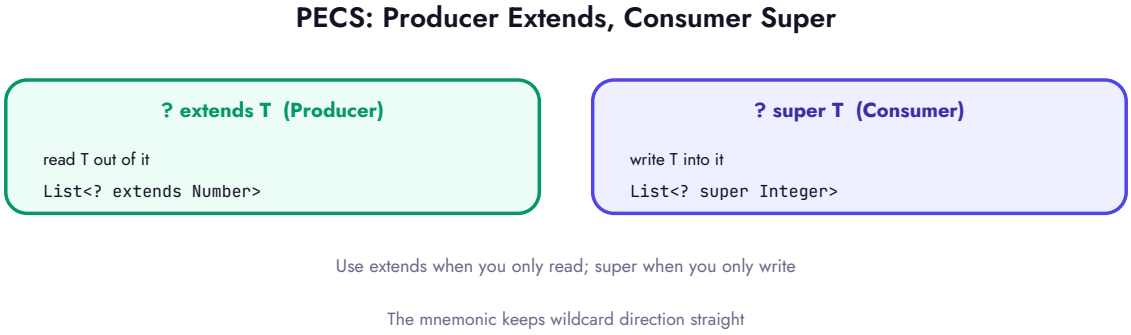
The problem wildcards solve

Generics are **invariant**: a `List<Integer>` is not a `List<Number>`, even though `Integer` is a `Number`. Wildcards reintroduce controlled flexibility.

```java
List<Integer> ints = List.of(1, 2, 3);
// void sum(List<Number> nums)  <- would NOT accept ints
void sum(List<? extends Number> nums) { }   // now it does
```

PECS

Use `? extends T` when you only **read** (the structure produces T), and `? super T` when you only **write** (the structure consumes T).



Producer Extends, Consumer Super — the mnemonic for wildcard direction.

```java
// Producer: we READ Numbers out -> extends
double total(List<? extends Number> src) {
    double t = 0; for (Number n : src) t += n.doubleValue(); return t;
}
// Consumer: we WRITE Integers in -> super
void fill(List<? super Integer> dst) {
    for (int i = 0; i < 5; i++) dst.add(i);
}
```

The unbounded wildcard

`List<?>` means 'a list of some unknown type'. You can read elements as `Object` and check size, but you cannot add anything except `null`.

Wildcard	You can
? extends T	read as T (produce)
? super T	write T (consume)
?	read as Object only

TIP

The JDK follows PECS everywhere — see `Collections.copy(dest, src)`, which takes `? super T` for the destination and `? extends T` for the source. When in doubt, mirror the standard library.

Read extends, write super

Wildcards make generic methods accept a family of related types. Remember PECS: use `extends` for things you read from and `super` for things you write to — and your APIs become both safe and flexible.

MODULE 2 · ADVANCED GENERICS

CHAPTER 07

Generic Methods & Type Inference

Writing methods parameterised on their own types, and how the compiler infers them.

A **generic method** declares its own type parameters, independent of any generic class. Combined with the compiler's type inference, generic methods give you reusable, type-safe utilities that read almost like ordinary calls.

Declaring a generic method

The type parameter list appears before the return type. The compiler usually infers it from the arguments, so callers rarely specify it.

java

```
static <T> List<T> repeat(T value, int n) {
    List<T> out = new ArrayList<>();
    for (int i = 0; i < n; i++) out.add(value);
    return out;
}
List<String> hellos = repeat("hi", 3);   // T inferred as String
```

Multiple and bounded type parameters

java

```
static <K, V> Map<V, K> invert(Map<K, V> map) {
    Map<V, K> out = new HashMap<>();
    map.forEach((k, v) -> out.put(v, k));
    return out;
}

static <T extends Comparable<T>> T max(List<T> items) {
    return items.stream().max(Comparator.naturalOrder()).orElseThrow();
}
```

Type inference and the diamond

Inference lets you omit type arguments in many places: the diamond <> on the right of an assignment, and lambda parameter types. It reduces noise while keeping full safety.

java

```
Map<String, List<Integer>> m = new HashMap<>();   // diamond
list.sort((a, b) -> a.length() - b.length());     // types inferred
```

When inference needs help

Occasionally the compiler cannot infer a type — for example with a bare `Collections.emptyList()` in an ambiguous position. Supply an explicit type witness to guide it.

```java
List<String> empty = Collections.<String>emptyList();   // witness
```

NOTE

Prefer letting inference do the work; add explicit type arguments only when the compiler genuinely cannot resolve them. Over-specifying types adds noise without benefit.

Reusable and inferred

Generic methods bring type safety to standalone utilities, and inference keeps the call sites clean. Together they let you write flexible library-quality helpers that feel like ordinary Java.

© 2026 ByteMart Digital

MODULE 3 · FUNCTIONAL JAVA

CHAPTER 08

Functional Interfaces in Depth

The full functional toolkit — specialised, primitive, and custom functional interfaces.

Beyond the four you already know, `java.util.function` offers a rich family of functional interfaces — primitive-specialised, two-argument, and operator variants. Knowing them lets you express behaviour precisely and avoid needless boxing.

The core families

Interface	Shape
Function	T -> R
UnaryOperator	T -> T
BinaryOperator	(T,T) -> T
BiFunction	(T,U) -> R
Predicate	T -> boolean
Consumer / Supplier	T -> void / () -> T

Primitive specialisations

Generic functional interfaces box primitives, which costs performance in hot loops. Specialised variants like `IntFunction`, `ToIntFunction`, and `IntPredicate` avoid boxing entirely.

```java
IntUnaryOperator square = n -> n * n;        // no Integer boxing
ToIntFunction<String> len = String::length;
IntStream.rangeClosed(1, 5).map(square).sum();
```

Writing your own

Any interface with a single abstract method is a functional interface. Mark it `@FunctionalInterface` so the compiler enforces the one-method rule.

```java
@FunctionalInterface
interface Parser<T> {
    T parse(String input) throws ParseException;
}
Parser<Integer> p = Integer::parseInt;
```

Default methods on functional interfaces

Functional interfaces can carry default methods for composition — which is exactly how `Function.andThen` and `Predicate.and` are provided.

```java
Predicate<String> nonEmpty = s -> !s.isEmpty();
Predicate<String> shortStr = s -> s.length() < 10;
Predicate<String> valid = nonEmpty.and(shortStr);   // composed
```

TIP

Reach for a primitive-specialised interface (`IntPredicate`, `LongSupplier`) whenever you work with primitives in performance-sensitive code — it eliminates silent boxing costs.

Precise behaviour types

The functional interface family lets you type behaviour exactly — by arity, by primitive, by shape. Choosing the right one keeps code expressive and, for primitives, fast.

© 2026 ByteMart Digital

MODULE 3 · FUNCTIONAL JAVA

CHAPTER 09

Function Composition

Building complex behaviour by combining small functions with andThen, compose, and chaining.

Functional programming favours building large behaviour from small, composable pieces. Java's functional interfaces support **composition** — combining functions and predicates into new ones — letting you assemble pipelines declaratively.

Composing functions

`andThen` runs this function, then the next; `compose` runs the argument first. The order is the only difference.

java

```
Function<Integer, Integer> times2 = n -> n * 2;
Function<Integer, Integer> plus3 = n -> n + 3;

times2.andThen(plus3).apply(5);   // (5*2)+3 = 13
times2.compose(plus3).apply(5);   // (5+3)*2 = 16
```

Composing predicates

java

```
Predicate<Integer> positive = n -> n > 0;
Predicate<Integer> even = n -> n % 2 == 0;

positive.and(even).test(4);   // true
positive.or(even).test(-2);   // true
positive.negate().test(-1);   // true
```

Composing consumers

java

```
Consumer<String> log = System.out::println;
Consumer<String> audit = this::record;
Consumer<String> both = log.andThen(audit);
both.accept("event");   // runs both in order
```

Composition as a design tool

- Build behaviour from **small, testable** functions.
- Assemble pipelines **declaratively** instead of nesting calls.
- Reuse the same pieces in different combinations.
- Keep each function **pure** so composition stays predictable.

© 2026 ByteMart Digital

NOTE

Composition and streams complement each other: a composed `Function` is a natural argument to `map`, and a composed `Predicate` to `filter`.

Small pieces, combined

Function composition turns behaviour into building blocks you assemble on demand. Small pure functions combined with `andThen`, `and`, and `or` express complex logic clearly and reusably.

© 2026 ByteMart Digital

MODULE 3 · FUNCTIONAL JAVA

CHAPTER 10

Advanced Streams

10

flatMap, grouping, partitioning, and the operations that handle real-world data shapes.

Basic streams filter and map; advanced streams reshape data. `flatMap` flattens nested structures, grouping collectors build maps, and specialised operations handle the messy shapes real data arrives in.

flatMap: flattening nesting

`flatMap` turns each element into a stream and concatenates them — the tool for flattening lists of lists or expanding one element into many.

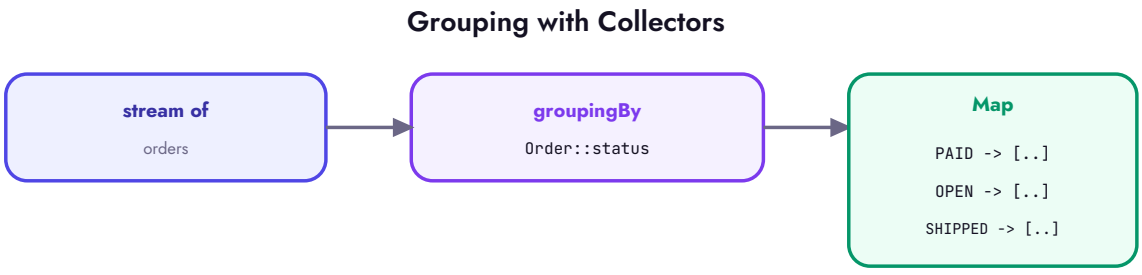
```java
List<List<Integer>> nested = List.of(List.of(1, 2), List.of(3, 4));
List<Integer> flat = nested.stream()
    .flatMap(List::stream)
    .toList();                        // [1, 2, 3, 4]
```

Grouping and partitioning

`groupingBy` builds a map from a classifier; `partitioningBy` splits into true/false buckets. A downstream collector aggregates each group.

```java
Map<Status, List<Order>> byStatus = orders.stream()
    .collect(Collectors.groupingBy(Order::status));

Map<Status, Long> counts = orders.stream()
    .collect(Collectors.groupingBy(Order::status, Collectors.counting()));
```



Collectors turn a stream into maps, sums, joins, and partitions

groupingBy turns a stream into a map keyed by a classifier function.

Primitive streams

`IntStream`, `LongStream`, and `DoubleStream` avoid boxing and add numeric operations like `sum`, `average`, and `summaryStatistics`.

```java
IntSummaryStatistics stats = orders.stream()
    .mapToInt(Order::total)
    .summaryStatistics();
// stats.getMax(), getAverage(), getSum(), getCount()
```

Useful terminal shapes

- `toMap(keyFn, valFn)` — build a map directly.
- `reduce` — fold to a single value with an identity and combiner.
- `peek` — observe elements mid-pipeline (debugging only).
- `takeWhile` / `dropWhile` — slice by a condition.

WATCH OUT

Avoid side effects inside stream operations — modifying external state from a `map` or `forEach` breaks under parallelism and hides bugs. Collect results instead.

Reshape, don't just filter

Advanced streams are about transforming data structure: flatten with `flatMap`, aggregate with grouping collectors, and stay boxing-free with primitive streams. This is where streams replace pages of loops.

© 2026 ByteMart Digital

MODULE 3 · FUNCTIONAL JAVA

CHAPTER 11

Custom Collectors

Building and combining collectors to aggregate streams into exactly the result you need.

Collectors are how streams produce results. Beyond the built-ins, you can combine them with downstream collectors, adapt them with `mapping` and `filtering`, and — when needed — build one from scratch.

Downstream collectors

The second argument to `groupingBy` is a downstream collector that aggregates each group — counting, summing, mapping, or joining.

```java
Map<Dept, Double> avgSalary = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::dept,
        Collectors.averagingDouble(Employee::salary)));

Map<Dept, List<String>> names = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::dept,
        Collectors.mapping(Employee::name, Collectors.toList())));
```

Adapting collectors

- `mapping(fn, downstream)` — transform before collecting.
- `filtering(pred, downstream)` — filter within a group.
- `collectingAndThen(c, fn)` — post-process the result.
- `teeing(c1, c2, merger)` — run two collectors and combine.

```java
// Immutable result via collectingAndThen
List<String> frozen = names.stream()
    .collect(Collectors.collectingAndThen(
        Collectors.toList(), List::copyOf));
```

Building a collector from scratch

`Collector.of` takes a supplier (new container), accumulator (add one element), combiner (merge two containers, for parallelism), and an optional finisher.

```java
Collector<String, ?, String> csv = Collector.of(
    StringBuilder::new,                        // supplier
    (sb, s) -> sb.append(s).append(','),       // accumulator
    StringBuilder::append,                     // combiner
    StringBuilder::toString);                  // finisher
```

TIP

Reach for a custom collector only when composition of the built-ins cannot express your aggregation. Ninety percent of the time, `groupingBy` plus a downstream collector is enough.

Aggregate exactly right

Collectors compose: group, then aggregate each group; adapt with `mapping` and `filtering`; post-process with `collectingAndThen`. Build one by hand only for the rare case the toolkit cannot reach.

MODULE 4 · CONCURRENCY FOUNDATIONS

CHAPTER 12

The Java Memory Model

Visibility, ordering, and the happens-before relationship that make concurrent code correct.

Concurrency bugs are often not about locking but about **visibility**: one thread's writes not being seen by another. The Java Memory Model (JMM) defines exactly when writes become visible, through the **happens-before** relationship. This is the theory beneath every correct concurrent program.

Why visibility is not free

Compilers and CPUs reorder instructions and cache values in registers for speed. Without synchronisation, a write by one thread may never become visible to another — or may appear out of order.

```java
// Broken: reader may loop forever
boolean running = true;
// Thread A: running = false;
// Thread B: while (running) { }   // may never see the change
```

happens-before

If action A **happens-before** action B, A's effects are visible to B. The JMM establishes this through synchronisation actions — locks, `volatile`, thread start/join, and more.

The happens-before Relationship



A volatile write, then read, guarantees A's earlier writes are visible to B

Without it, reordering and caching can hide updates between threads

A volatile write then read establishes happens-before, making earlier writes visible.

volatile

A `volatile` field guarantees visibility and ordering: writes are flushed and reads are fresh, and it prevents reordering across the access. It does not make compound operations atomic.

```java
volatile boolean running = true;   // now the loop above works
// but volatile does NOT fix count++ (read-modify-write)
```

Sources of happens-before

- Unlocking a monitor happens-before locking it.
- A `volatile` write happens-before a later read of it.
- `Thread.start()` happens-before the thread's run.
- A thread's actions happen-before another's return from `join()`.

WATCH OUT

`volatile` guarantees visibility, not atomicity. For counters and compound updates use atomic classes or locks — `volatile` alone cannot make `x++` safe.

Visibility is the real hazard

Correct concurrency is about establishing happens-before, not just avoiding two threads at once. Use `volatile` for simple visibility, locks and atomics for compound updates, and reason in terms of what is guaranteed to be seen.

© 2026 ByteMart Digital

MODULE 4 · CONCURRENCY FOUNDATIONS

CHAPTER 13

Locks & Synchronizers

Explicit locks beyond synchronized — ReentrantLock, read-write locks, and StampedLock.

`synchronized` is simple but rigid. The `java.util.concurrent.locks` package offers explicit locks with features synchronized cannot provide: timed and interruptible acquisition, fairness, separate read/write access, and optimistic reads.

ReentrantLock

A `ReentrantLock` is an explicit, reentrant mutual-exclusion lock. Always unlock in a `finally` block so the lock is released even on exception.

java

```
private final ReentrantLock lock = new ReentrantLock();

void update() {
    lock.lock();
    try { /* critical section */ }
    finally { lock.unlock(); }   // always release
}
```

What explicit locks add

- `tryLock()` — acquire only if free (avoid deadlock).
- `tryLock(timeout)` — give up after a while.
- `lockInterruptibly()` — respond to interruption.
- Optional **fairness** — grant the lock in arrival order.

ReadWriteLock

A `ReadWriteLock` allows many concurrent readers but only one writer — ideal for data read far more often than written.

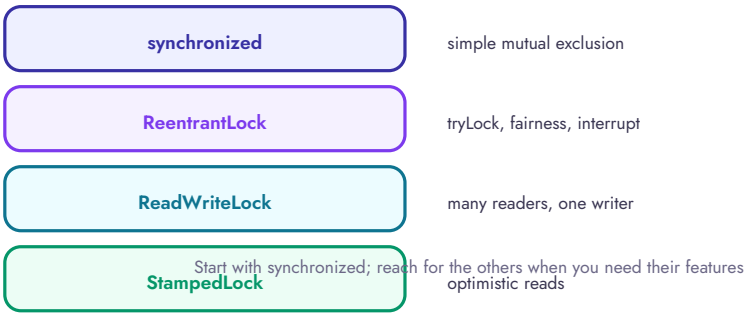
java

```
private final ReadWriteLock rw = new ReentrantReadWriteLock();
String read()  { rw.readLock().lock();  try { return data; } finally { rw.readLock().unlock();
void write(String v) { rw.writeLock().lock(); try { data = v; } finally { rw.writeLock().unlocl
```

StampedLock and optimistic reads

`StampedLock` adds an **optimistic read** mode: read without locking, then validate. If a write intervened, fall back to a real read lock. It can dramatically cut contention on read-heavy data.

Choosing a Lock



Pick the lock that fits: synchronized, ReentrantLock, ReadWriteLock, or StampedLock.

WATCH OUT

Explicit locks are unforgiving: forget the `finally` unlock and you leak the lock, freezing every waiting thread. Prefer `synchronized` unless you need a feature only explicit locks provide.

The right lock for the access

Use `synchronized` by default; reach for `ReentrantLock` when you need timed or interruptible acquisition, read-write locks for read-heavy data, and `StampedLock` to squeeze out optimistic reads.

MODULE 4 · CONCURRENCY FOUNDATIONS

CHAPTER 14

Concurrent Collections & Atomics

Thread-safe collections and lock-free atomic variables for high-performance shared state.

Wrapping a plain collection in a lock serialises every access and kills throughput. The concurrent collections are engineered for safe, high-concurrency access, and atomic variables provide lock-free updates for single values.

ConcurrentHashMap

`ConcurrentHashMap` allows concurrent reads and fine-grained concurrent writes. Its atomic methods — `compute`, `merge`, `putIfAbsent` — let you update safely without external locking.

```java
ConcurrentHashMap<String, Integer> counts = new ConcurrentHashMap<>();
counts.merge(word, 1, Integer::sum);          // atomic increment
counts.computeIfAbsent(key, k -> loadDefault(k));
```

Other concurrent collections

Type	Use for
ConcurrentHashMap	concurrent map access
CopyOnWriteArrayList	many reads, rare writes
ConcurrentLinkedQueue	lock-free queue
BlockingQueue	producer-consumer handoff

BlockingQueue for handoff

A `BlockingQueue` is the classic producer-consumer channel: producers `put` (blocking if full), consumers `take` (blocking if empty). No manual wait/notify needed.

```java
BlockingQueue<Task> queue = new LinkedBlockingQueue<>(100);
// producer: queue.put(task);
// consumer: Task t = queue.take();
```

Atomic variables

`AtomicInteger`, `AtomicLong`, and `AtomicReference` use hardware compare-and-swap for lock-free updates — faster than locks for single-variable counters and flags.

© 2026 ByteMart Digital

```java
AtomicInteger id = new AtomicInteger();
int next = id.incrementAndGet();       // atomic, lock-free

AtomicReference<State> state = new AtomicReference<>(INIT);
state.compareAndSet(INIT, RUNNING);    // CAS
```

TIP

For high-contention counters, prefer `LongAdder` over `AtomicLong` — it spreads updates across cells and reads the total on demand, reducing contention dramatically.

Concurrency-designed, not locked

Concurrent collections and atomics are built for many threads, not retrofitted with a single lock. Use `ConcurrentHashMap`, `BlockingQueue`, and atomic classes instead of guarding plain collections yourself.

© 2026 ByteMart Digital

MODULE 4 · CONCURRENCY FOUNDATIONS

CHAPTER 15

Thread Coordination

Coordinating threads with latches, barriers, semaphores, and phasers.

Beyond mutual exclusion, threads often need to **coordinate**: wait for others to finish, meet at a barrier, or limit how many run at once. The synchronizers in `java.util.concurrent` express these patterns cleanly, replacing fragile `wait/notify` code.

CountDownLatch: wait for completion

A `CountDownLatch` makes threads wait until a counter reaches zero — perfect for 'wait until N tasks are done'. It is one-shot.

```java
CountDownLatch done = new CountDownLatch(3);
for (int i = 0; i < 3; i++)
    pool.submit(() -> { work(); done.countDown(); });
done.await();                      // blocks until all three finish
System.out.println("all done");
```

CyclicBarrier: meet and repeat

A `CyclicBarrier` makes a set of threads wait for each other at a common point, then proceed together — and it resets, so it works across repeated phases.

```java
CyclicBarrier barrier = new CyclicBarrier(4, () -> mergeResults());
// each worker: compute(); barrier.await();  // all meet, then merge runs
```

Semaphore: limit concurrency

A `Semaphore` holds a set of permits, letting only that many threads proceed at once — ideal for rate-limiting access to a scarce resource.

```java
Semaphore permits = new Semaphore(5);   // at most 5 at a time
permits.acquire();
try { callExternalApi(); }
finally { permits.release(); }
```

Choosing a synchronizer

Need	Use
wait for N to finish	CountDownLatch

Need	Use
threads meet, then repeat	CyclicBarrier
limit concurrent access	Semaphore
dynamic multi-phase	Phaser

Coordinate, don't hand-roll

Latches, barriers, semaphores, and phasers express common coordination patterns correctly and readably. Reach for them instead of hand-written `wait/notify`, which is easy to get subtly wrong.

© 2026 ByteMart Digital

MODULE 5 · ASYNC & PARALLEL

CHAPTER 16

CompletableFuture

Composing asynchronous work into non-blocking pipelines that chain, combine, and handle errors.

`CompletableFuture` is Java's tool for asynchronous programming. Instead of blocking on a result or nesting callbacks, you describe a pipeline of stages that run as each dependency completes — composable, non-blocking, and with built-in error handling.

Starting async work

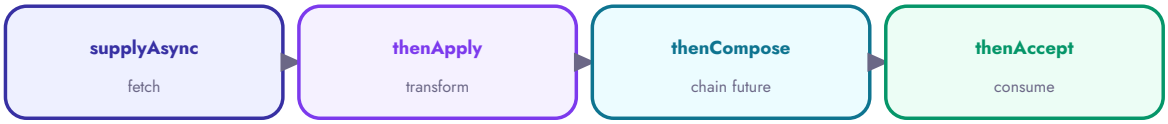
```java
CompletableFuture<String> f =
    CompletableFuture.supplyAsync(() -> fetchUser(id));   // runs on a pool
// supplyAsync returns a value; runAsync returns void
```

Chaining stages

`thenApply` transforms the result; `thenCompose` chains another future (flat-map); `thenAccept` consumes it. Each runs when the previous completes.

```java
CompletableFuture.supplyAsync(() -> loadOrder(id))
    .thenApply(Order::total)                 // transform
    .thenCompose(total -> applyDiscount(total))  // chain a future
    .thenAccept(System.out::println);        // consume
```

Composing Async with CompletableFuture



Each stage runs when the previous completes — no blocking, no callbacks nested

Each stage runs when the previous completes — non-blocking, no nested callbacks.

Combining futures

`thenCombine` merges two independent futures; `allOf` waits for many; `anyOf` completes on the first.

```java
CompletableFuture<Integer> a = supplyAsync(this::loadA);
CompletableFuture<Integer> b = supplyAsync(this::loadB);
a.thenCombine(b, Integer::sum)              // run both, then add
 .thenAccept(System.out::println);

CompletableFuture.allOf(a, b).join();       // wait for both
```

Handling errors

Exceptions flow down the chain. `exceptionally` supplies a fallback, and `handle` receives both result and error.

```java
supplyAsync(this::risky)
    .exceptionally(ex -> "fallback")        // recover
    .thenAccept(System.out::println);
```

WATCH OUT

Do not call `get()` or `join()` in the middle of a pipeline — it blocks and defeats the purpose. Compose with the `then*` methods and block only at the very edge, if at all.

Describe async as a pipeline

`CompletableFuture` turns callback spaghetti into a readable chain of stages that transform, combine, and recover. Compose non-blocking work; block only at the boundary.

© 2026 ByteMart Digital

MODULE 5 · ASYNC & PARALLEL

CHAPTER 17

Executors & Fork/Join

Managing thread pools with executors and parallelising recursive work with the fork/join framework.

Creating threads by hand does not scale. **Executors** manage pools of reusable threads you submit tasks to, and the **fork/join** framework parallelises divide-and-conquer work with work-stealing. Together they are the foundation of scalable server-side Java.

Executor types

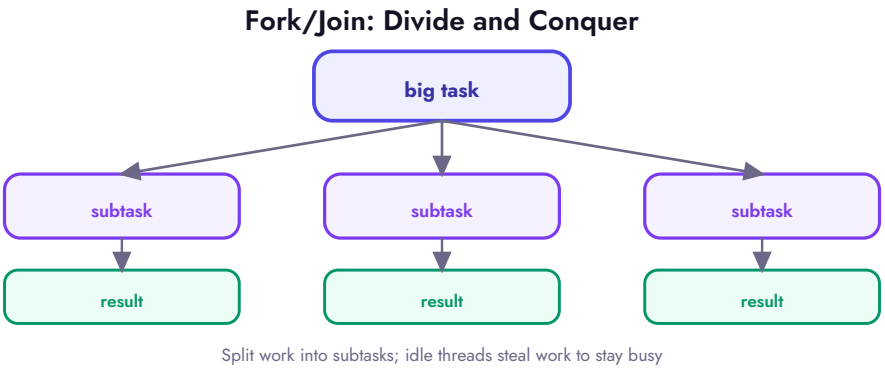
Factory	Gives you
newFixedThreadPool(n)	n reusable threads
newCachedThreadPool()	grows/shrinks on demand
newScheduledThreadPool	delayed / periodic tasks
newVirtualThreadPerTask	one virtual thread per task

java

```
ExecutorService pool = Executors.newFixedThreadPool(8);
List<Future<Integer>> futures = tasks.stream()
    .map(t -> pool.submit(t))
    .toList();
for (var f : futures) total += f.get();
pool.shutdown();
```

The fork/join framework

Fork/join splits a task into subtasks (`fork`), then combines their results (`join`). Idle threads **steal** work from busy ones, keeping all cores busy.



Split work into subtasks; idle threads steal work to stay busy.

```java
class SumTask extends RecursiveTask<Long> {
    final long[] a; final int lo, hi;
    SumTask(long[] a, int lo, int hi) { this.a=a; this.lo=lo; this.hi=hi; }
    protected Long compute() {
        if (hi - lo <= 1000) {                // base case
            long s = 0; for (int i=lo;i<hi;i++) s += a[i]; return s;
        }
        int mid = (lo + hi) >>> 1;
        SumTask left = new SumTask(a, lo, mid);
        left.fork();                          // async
        SumTask right = new SumTask(a, mid, hi);
        return right.compute() + left.join();  // combine
    }
}
```

Sizing pools

- **CPU-bound** work — pool size near the core count.
- **I/O-bound** work — more threads, since they mostly wait (or use virtual threads).
- Always **shut down** executors to release their threads.
- The common fork/join pool backs parallel streams too.

Pools, not raw threads

Executors reuse threads and decouple task submission from execution; fork/join parallelises recursive work with work-stealing. Submit tasks to a sized pool rather than spawning threads yourself.

MODULE 5 · ASYNC & PARALLEL

CHAPTER 18

Parallel Streams

When and how to parallelise stream pipelines — and the pitfalls that make them slower.

A parallel stream splits its work across the fork/join common pool, potentially using every core. It is a single method call — but it is not free speed. Parallel streams help only for the right workloads, and hurt for the wrong ones.

Going parallel

java

```
long count = orders.parallelStream()
    .filter(Order::isPaid)
    .mapToLong(Order::total)
    .sum();
// or: stream().parallel()
```

When parallel helps

- **Large** data sets — enough work to outweigh split/merge overhead.
- **CPU-bound**, independent per-element work.
- Cheaply **splittable** sources (arrays, `ArrayList`, ranges).
- **Stateless**, side-effect-free operations.

When it hurts

- Small collections — overhead dominates.
- Poorly splittable sources (`LinkedList`, streams from I/O).
- Cheap per-element work — nothing to gain.
- Ordered operations that force merging, or shared mutable state.

The shared-pool caveat

Parallel streams use the shared fork/join common pool. A slow or blocking parallel stream can starve every other parallel stream in the JVM — never do blocking I/O inside one.

WATCH OUT

Never call blocking operations (network, disk) inside a parallel stream. It ties up the shared common pool and can stall the whole application. Use an executor or virtual threads for I/O instead.

TIP

Measure, do not guess. Benchmark a parallel stream against the sequential version with a real workload — the crossover point depends on data size, element cost, and hardware.

© 2026 ByteMart Digital

Parallel is a tuning choice, not a default

Parallel streams can use every core, but only pay off for large, CPU-bound, splittable work with no side effects. Profile before parallelising — the wrong workload gets slower, not faster.

© 2026 ByteMart Digital

MODULE 6 · VIRTUAL THREADS & LOOM

CHAPTER 19

Virtual Threads

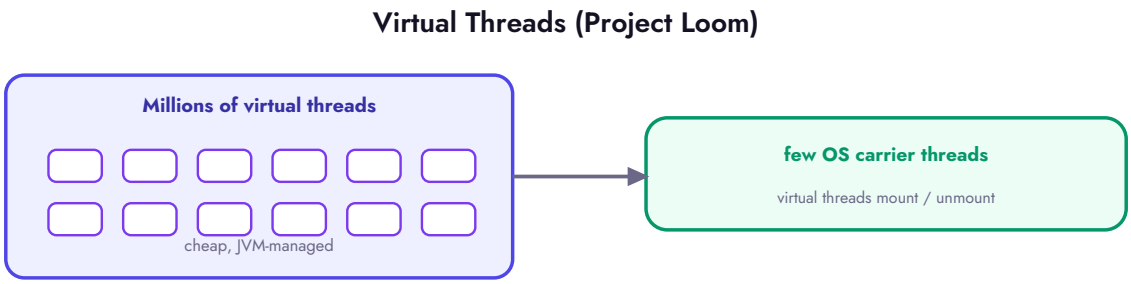
Lightweight, JVM-managed threads that make blocking code scale to millions of concurrent tasks.

Virtual threads (from Project Loom, standard since Java 21) are lightweight threads managed by the JVM rather than the OS. They let you write simple, blocking, thread-per-request code that scales to millions of concurrent tasks — the async performance without the async complexity.

`Virtual thread`
A lightweight thread scheduled by the JVM onto a small pool of OS 'carrier' threads, so blocking one is cheap and vast numbers can run at once.

The problem they solve

Platform (OS) threads are expensive — a server can run a few thousand at most. To scale, developers turned to complex asynchronous, callback-heavy code. Virtual threads let blocking code scale instead.



Blocking a virtual thread is cheap — it unmounts instead of holding an OS thread

Millions of cheap virtual threads mount and unmount on a few OS carrier threads.

Creating virtual threads

java

```
Thread.startVirtualThread(() -> handle(request));

// Best: an executor that creates one virtual thread per task
try (var exec = Executors.newVirtualThreadPerTaskExecutor()) {
    for (var req : requests)
        exec.submit(() -> handle(req));   // millions is fine
}
```

How they scale

When a virtual thread blocks (on I/O, a lock, or sleep), it **unmounts** from its carrier thread, freeing that OS thread for other work. It remounts when the blocking call returns. Blocking becomes cheap.

- Write straightforward **blocking** code — no callbacks or reactive plumbing.

- One virtual thread **per task/request** is the intended model.
- They are cheap to create and discard — do not pool them.
- Existing thread-based APIs work unchanged.

WATCH OUT

Do not pool virtual threads or use thread-locals heavily with them — both patterns assume expensive threads. Create a fresh virtual thread per task, which is exactly what the per-task executor does.

Blocking code that scales

Virtual threads make the simple thread-per-task model scale to enormous concurrency by unmounting on block. They let you keep writing clear, sequential code and get async-level throughput for free.

MODULE 6 · VIRTUAL THREADS & LOOM

CHAPTER 20

Structured Concurrency

Treating a group of concurrent tasks as one unit — for clean cancellation and error handling.

When one task spawns several concurrent subtasks, managing their lifetimes, errors, and cancellation by hand is error-prone. **Structured concurrency** ties subtasks to a scope: they start together, and the scope ensures they all complete or are cancelled together — no leaks, no orphans.

> **`Structured concurrency`**
> An approach that binds the lifetime of concurrent subtasks to a syntactic scope, so their creation, joining, and cancellation are managed as one unit.

The problem with unstructured tasks

Submit two tasks to an executor and you must remember to join both, cancel the sibling if one fails, and handle partial failures. Miss a case and threads leak or errors are swallowed. Structured concurrency makes the scope enforce correctness.

A structured scope

`StructuredTaskScope` forks subtasks, joins them together, and propagates cancellation. If one fails, the others are cancelled automatically.

```java
try (var scope = new StructuredTaskScope.ShutdownOnFailure()) {
    var user  = scope.fork(() -> fetchUser(id));    // subtask 1
    var order = scope.fork(() -> fetchOrder(id));   // subtask 2
    scope.join();                 // wait for both
    scope.throwIfFailed();        // propagate any failure
    return combine(user.get(), order.get());
}   // scope guarantees both are done or cancelled
```

What the scope guarantees

- Subtasks cannot outlive the scope — no leaked threads.
- If one subtask fails, siblings are **cancelled**.
- Errors propagate to the parent, not silently vanish.
- The code reads top-to-bottom, mirroring the task structure.

> **NOTE**
> Structured concurrency builds on virtual threads: each forked subtask runs on a cheap virtual thread, so forking many is inexpensive. The two features are designed to work together.

Concurrency with a clear shape

Structured concurrency makes a group of subtasks behave like a single call: they finish together, failures cancel siblings, and nothing leaks. It brings the discipline of ordinary code structure to concurrent work.

© 2026 ByteMart Digital

MODULE 7 · JVM INTERNALS & PERFORMANCE

CHAPTER 21

Class Loading

How the JVM finds, links, and initialises classes — and the class-loader hierarchy.

Before your code runs, the JVM must load its classes. **Class loading** is a lazy, multi-stage process handled by a hierarchy of class loaders. Understanding it demystifies `ClassNotFoundException`, static initialisation order, and how frameworks load plugins.

The lifecycle

A class goes through **loading** (read the bytes), **linking** (verify, prepare static fields, resolve references), and **initialisation** (run static initialisers). It happens lazily — on first active use, not at startup.



Loading, linking, and initialisation — the JVM brings classes in lazily on first use.

The class-loader hierarchy

Loaders form a parent chain: the **bootstrap** loader (core JDK), the **platform** loader, and the **application** loader (your classpath). Each delegates to its parent first.

Loader	Loads
Bootstrap	core JDK classes
Platform	JDK modules / extensions
Application	your classpath / modules

The delegation model

By default a loader asks its parent to load a class before trying itself. This **parent-first delegation** prevents core classes from being overridden and keeps class identity consistent.

Custom class loaders

Frameworks and app servers write custom loaders to isolate plugins, hot-reload code, or load classes from unusual sources. A class's identity is its name plus its loader — the same bytes loaded by two loaders are two distinct types.

© 2026 ByteMart Digital

WATCH OUT

Two classes with the same name loaded by different class loaders are not the same type — a cast between them throws `ClassCastException`. This is a classic source of confusing errors in app servers and plugin systems.

Lazy, delegated, identity-bearing

Classes load on first use through a delegating loader hierarchy, and a class's identity includes its loader. This explains initialisation order, isolation between plugins, and a whole family of loader-related errors.

© 2026 ByteMart Digital

MODULE 7 · JVM INTERNALS & PERFORMANCE

CHAPTER 22

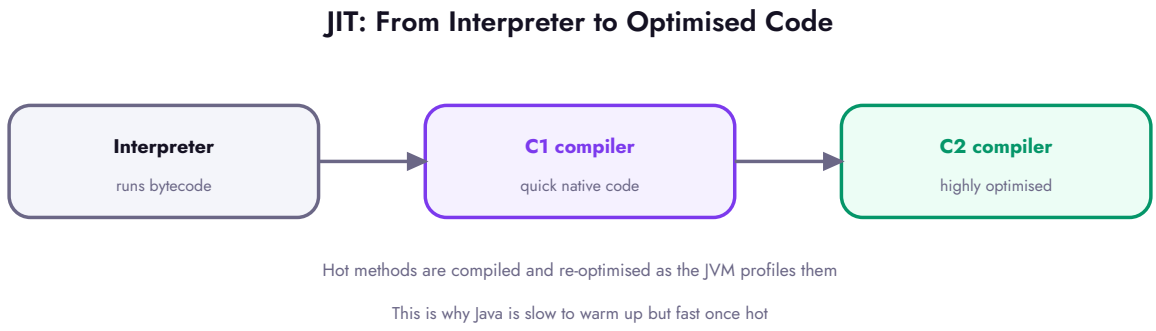
JIT Compilation

How the JVM turns hot bytecode into optimised native code at runtime.

Java starts by interpreting bytecode, then compiles the **hot** parts to native code as the program runs — **just-in-time** compilation. This is why Java is slow to warm up but competitive with native code once hot, and why microbenchmarks are so easy to get wrong.

Interpretation, then compilation

The JVM interprets bytecode at first, profiling which methods run often. Hot methods are compiled by the C1 (fast) and C2 (highly optimising) compilers, a scheme called tiered compilation.



Hot methods move from interpreter to C1 to C2 as the JVM profiles them.

Profile-guided optimisation

Because it compiles at runtime, the JIT uses **actual** profile data — which branches are taken, which types occur — to optimise aggressively in ways an ahead-of-time compiler cannot.

- **Inlining** — replace hot method calls with their bodies.
- **Escape analysis** — allocate non-escaping objects on the stack.
- **Loop optimisations** — unrolling, hoisting, bounds-check elimination.
- **Speculative** optimisation — assume common cases, deoptimise if wrong.

Deoptimisation

When a speculative assumption breaks — a rare type finally appears — the JVM **deoptimises** back to the interpreter and recompiles. This adaptability is a strength, not a failure.

Why warm-up matters for benchmarks

WATCH OUT

Timing code on its first run measures the interpreter, not the optimised version. Never benchmark without a warm-up phase — this is exactly why JMH (next chapters) exists.

Optimised at runtime

The JIT compiles hot code using real profile data, applying optimisations an offline compiler cannot. It explains Java's warm-up cost, its steady-state speed, and why honest benchmarking requires care.

© 2026 ByteMart Digital

MODULE 7 · JVM INTERNALS & PERFORMANCE

CHAPTER 23

Garbage Collection Deep Dive

Modern collectors, their trade-offs, and how to reason about GC-related performance.

Garbage collection frees you from manual memory management, but its behaviour shapes an application's latency and throughput. Modern Java offers several collectors with different trade-offs — choosing and reasoning about them is a key advanced skill.

The core cycle

At heart, GC **marks** reachable objects, **sweeps** the unreachable, and often **compacts** to reduce fragmentation. Modern collectors do much of this concurrently to keep pauses short.

Mark - Sweep - Compact



G1, ZGC, and Shenandoah do this concurrently for low pauses

Mark reachable objects, sweep garbage, and compact — done concurrently by modern GCs.

The collectors

Collector	Optimised for
G1 (default)	balanced, predictable pauses
Parallel	raw throughput (batch jobs)
ZGC	very low latency, huge heaps
Shenandoah	low pause, concurrent compaction

Throughput vs latency

The central trade-off: collectors that do more work concurrently keep pauses short (low latency) but use more CPU; stop-the-world collectors maximise throughput but pause longer. Pick based on whether your app is latency- or throughput-sensitive.

Reasoning about GC

- **Allocate less** — the cheapest GC is the one that does not run.
- Most objects die young; short-lived garbage is cheap to collect.
- Watch for **promotion** of medium-lived objects into the old generation.

© 2026 ByteMart Digital

- Enable GC logs to see pause times and frequency before tuning.

TIP

Do not tune GC blindly. Measure with GC logging and a profiler first; often the real fix is reducing allocation, not changing collector flags. Choose the collector to match your latency or throughput goal, then tune from data.

Trade-offs, not magic

Every collector balances pause time against throughput and CPU. Understand the young/old split, pick a collector for your latency or throughput goal, and reduce allocation before reaching for tuning flags.

© 2026 ByteMart Digital

MODULE 7 · JVM INTERNALS & PERFORMANCE

CHAPTER 24

Memory & Off-Heap

Where memory really goes — heap, metaspace, thread stacks, and direct off-heap buffers.

The heap is only part of a JVM's memory footprint. Class metadata, thread stacks, code caches, and off-heap buffers all consume memory the GC does not manage. Knowing the full picture is essential for sizing containers and diagnosing out-of-memory errors.

The memory regions

Region	Holds
Heap	objects (GC-managed)
Metaspace	class metadata (native memory)
Thread stacks	frames, locals per thread
Code cache	JIT-compiled native code
Direct buffers	off-heap NIO memory

Off-heap memory

A `ByteBuffer.allocateDirect` allocates memory **outside** the heap. It avoids copies for I/O and is not subject to GC pauses — but you must manage its lifetime, and it does not count against the heap size.

```java
ByteBuffer direct = ByteBuffer.allocateDirect(1 << 20);   // 1 MB off-heap
// great for high-throughput I/O; freed when the buffer is collected
```

Diagnosing memory problems

- `OutOfMemoryError: Java heap space` — too many live objects; take a heap dump.
- `OutOfMemoryError: Metaspace` — class-loading leak (common in redeploys).
- Native memory growth with a flat heap — suspect direct buffers.
- Use heap dumps and a profiler to find the retaining references.

WATCH OUT

In containers, set the heap explicitly and remember that non-heap regions add to the total. A container killed for exceeding its memory limit often has a healthy heap but large metaspace, stacks, or direct buffers.

Heap is not the whole story

A JVM uses heap, metaspace, stacks, code cache, and off-heap buffers. Sizing and diagnosing memory correctly — especially in containers — means accounting for all of them, not just the heap.

© 2026 ByteMart Digital

MODULE 7 · JVM INTERNALS & PERFORMANCE

CHAPTER 25

Benchmarking with JMH

Measuring Java performance honestly with the Java Microbenchmark Harness.

Timing code with `System.nanoTime()` gives misleading numbers: the JIT warms up, dead code gets eliminated, and the JVM optimises your benchmark away. The **Java Microbenchmark Harness (JMH)** is the standard tool for measuring Java performance correctly.

Why naive benchmarks lie

- **Warm-up** — the first runs measure the interpreter, not compiled code.
- **Dead-code elimination** — the JIT removes results you never use.
- **Constant folding** — it precomputes work on constant inputs.
- **On-stack replacement** and GC noise skew short measurements.

A JMH benchmark

```java
@BenchmarkMode(Mode.AverageTime)
@OutputTimeUnit(TimeUnit.NANOSECONDS)
@State(Scope.Thread)
public class ParseBench {
    String input = "12345";

    @Benchmark
    public int parse() {
        return Integer.parseInt(input);   // measured method
    }
}
```

Defeating the optimiser

Return results or hand them to a `Blackhole` so the JIT cannot eliminate them, and take inputs from `@State` so it cannot constant-fold. JMH also runs proper warm-up and measurement iterations for you.

```java
@Benchmark
public void consume(Blackhole bh) {
    bh.consume(compute());   // prevents dead-code elimination
}
```

Reading results

- Report an average or throughput **with error bars**, not a single number.
- Compare alternatives in the **same** run on the same machine.
- Beware differences smaller than the measurement error.

© 2026 ByteMart Digital

- Micro-results do not always predict whole-application performance.

Measure, don't guess

The JVM makes intuitive benchmarking wrong. JMH handles warm-up, prevents the optimiser from cheating, and reports statistically sound numbers — use it whenever a performance claim actually matters.

MODULE 8 · ADVANCED TECHNIQUES

CHAPTER 26

Reflection & Method Handles

Inspecting and invoking code at runtime — and the faster, safer method-handle alternative.

Reflection lets code examine and invoke classes, methods, and fields at runtime — the engine behind frameworks like Spring and Jackson. **Method handles** are a newer, faster, more type-safe alternative for the same dynamic invocation needs.

Reflection recap

java

```
Class<?> clazz = Class.forName("com.bytemart.Service");
Object instance = clazz.getDeclaredConstructor().newInstance();
Method m = clazz.getMethod("process", String.class);
m.invoke(instance, "input");     // dynamic call
```

Reading annotations and fields

java

```
for (Field f : clazz.getDeclaredFields()) {
    if (f.isAnnotationPresent(Inject.class)) {
        f.setAccessible(true);
        f.set(instance, resolve(f.getType()));   // DI by reflection
    }
}
```

Method handles

A `MethodHandle` is a typed, directly-invokable reference to a method, obtained through a `Lookup`. It is faster than reflective `invoke` and checks types at link time.

java

```
MethodHandles.Lookup lookup = MethodHandles.lookup();
MethodType type = MethodType.methodType(int.class, int.class);
MethodHandle sq = lookup.findVirtual(Math.class, "abs", type);
int r = (int) sq.invokeExact(-5);   // 5
```

Reflection vs method handles

	Reflection	Method Handles
Speed	slower	faster
Type safety	runtime	link time

© 2026 ByteMart Digital

	Reflection	Method Handles
Ease	simple	lower-level

WATCH OUT

Reflection bypasses compile-time checks and access control (`setAccessible`), and it is slow. Use it for frameworks and tooling — never for hot paths or ordinary application logic where direct calls are clearer and safer.

Dynamic, but deliberate

Reflection and method handles let code act on types discovered at runtime — the basis of DI, ORMs, and serialisers. Prefer method handles for performance and safety, and keep dynamic invocation to the framework layer.

MODULE 8 · ADVANCED TECHNIQUES

CHAPTER 27

Annotation Processing

Generating code and enforcing rules at compile time by processing annotations.

Annotations are metadata; **annotation processors** act on that metadata at compile time. They can validate code, generate new source files, and build boilerplate automatically — the technique behind Lombok, MapStruct, and Dagger.

Compile-time vs runtime

Runtime annotations are read via reflection while the program runs. **Compile-time** processing happens during `javac`, before your code ships — catching errors earlier and generating code with zero runtime cost.

How processors work

A processor extends `AbstractProcessor`, declares which annotations it handles, and is invoked by the compiler with the annotated elements. It can emit diagnostics or generate new files.

java

```
@SupportedAnnotationTypes("com.bytemart.Builder")
public class BuilderProcessor extends AbstractProcessor {
    public boolean process(Set<? extends TypeElement> annotations,
                           RoundEnvironment env) {
        for (Element e : env.getElementsAnnotatedWith(Builder.class)) {
            generateBuilderFor(e);   // write a new .java file
        }
        return true;
    }
}
```

Generating source

Processors write files via the `Filer`. Libraries like JavaPoet make building well-formed Java source far easier than string concatenation.

Where you see it

- **Lombok** — generates getters, builders, and constructors.
- **MapStruct** — generates type-safe mappers between objects.
- **Dagger** — generates dependency-injection code at compile time.
- **Immutables / AutoValue** — generate value classes.

> **NOTE**
> Compile-time generation trades a little build complexity for zero runtime overhead and earlier error detection — the opposite trade-off from reflection-based frameworks.

© 2026 ByteMart Digital

Metadata at compile time

Annotation processors turn annotations into validation and generated code during compilation. They power the libraries that eliminate boilerplate — with no runtime cost and errors caught before you ship.

© 2026 ByteMart Digital

MODULE 8 · ADVANCED TECHNIQUES

CHAPTER 28

The Module System (JPMS)

Strong encapsulation and reliable dependencies with the Java Platform Module System.

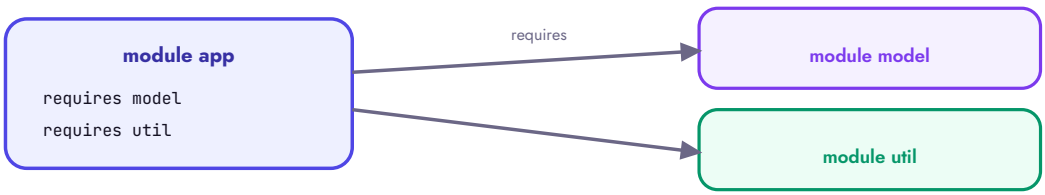
The Java Platform Module System (JPMS), introduced in Java 9, adds a layer above packages: **modules** that explicitly declare what they require and what they expose. It brings strong encapsulation and reliable configuration to large applications and to the JDK itself.

> **`Module`**
> A named, self-describing collection of packages that declares its dependencies (`requires`) and the packages it makes public (`exports`) in a `module-info.java` file.

module-info.java

```java
module com.bytemart.app {
    requires com.bytemart.model;      // dependency
    requires transitive java.sql;     // re-exported to consumers
    exports com.bytemart.app.api;     // public package
    // everything not exported is strongly encapsulated
}
```

The Module System (JPMS)



Modules declare what they require and export — strong encapsulation

Modules declare what they require and export, forming an explicit dependency graph.

Strong encapsulation

A package not `exportsed` is invisible outside its module — even to reflection. This is stronger than `public`, which was accessible to anyone on the classpath. Internals stay truly internal.

Key directives

Directive	Meaning
requires M	depend on module M
requires transitive M	re-export M to consumers

Directive	Meaning
exports P	make package P public
opens P	allow deep reflection on P

Reliable configuration

At startup the module system verifies the whole graph: every required module is present and there are no split packages or cycles. Missing dependencies fail fast, not deep into a run.

WATCH OUT

Adopting modules is optional and can be disruptive for apps built on reflective frameworks. Many projects still run on the classpath; introduce modules deliberately, starting with libraries that benefit most from encapsulation.

Explicit dependencies, real encapsulation

JPMS makes dependencies and exposed API explicit, and hides everything else — even from reflection. It is how the JDK is structured and a powerful tool for keeping large systems' internals genuinely internal.

© 2026 ByteMart Digital

MODULE 8 · ADVANCED TECHNIQUES

CHAPTER 29

Serialization

Turning objects into bytes and back — the built-in mechanism, its dangers, and safer alternatives.

Serialization converts an object graph into bytes for storage or transmission, and back again. Java has a built-in mechanism, but it carries real security and maintenance risks — so understanding both how it works and why modern code often avoids it is essential.

Built-in serialization

A class that implements the marker interface `Serializable` can be written by `ObjectOutputStream` and read by `ObjectInputStream`. The whole reachable object graph is serialised.

```java
class Order implements Serializable {
    private static final long serialVersionUID = 1L;
    private String id;
    private transient String cache;   // skipped by serialization
}
```

serialVersionUID and transient

- `serialVersionUID` versions the class; a mismatch fails deserialization.
- `transient` fields are skipped — for caches and secrets.
- Static fields are not serialised (they belong to the class).
- Changing a class can break reading old serialised data.

The security problem

WATCH OUT

Deserializing untrusted data is dangerous: crafted byte streams can trigger arbitrary code execution through gadget chains. **Never deserialize data you do not fully trust** — this has caused many severe vulnerabilities.

Safer alternatives

- **JSON** (Jackson, Gson) — human-readable, language-neutral, safe by default.
- **Protocol Buffers** / **Avro** — compact, schema-driven, cross-language.
- Explicit, hand-written conversion for full control.
- If you must use Java serialization, apply serialization filters.

Prefer explicit formats

Java's built-in serialization is convenient but fragile and unsafe on untrusted input. For most systems, a schema-based or JSON format is safer, more portable, and more maintainable — reserve native serialization for trusted, controlled cases.

© 2026 ByteMart Digital

© 2026 ByteMart Digital

MODULE 8 · ADVANCED TECHNIQUES

CHAPTER 30

Design Patterns in Modern Java

Classic patterns reimagined with lambdas, records, sealed types, and the modern language.

Design patterns are reusable solutions to recurring problems. In modern Java, many classic patterns become far simpler — lambdas replace single-method strategy classes, records model value objects, and sealed types express state machines. Knowing the modern form matters as much as the pattern itself.

Strategy, with a lambda

The Strategy pattern — swapping an algorithm at runtime — no longer needs a class per strategy. A functional interface plus lambdas expresses it directly.

```java
// Strategy is just a Comparator here
List<Order> orders = ...;
orders.sort(Comparator.comparing(Order::total));       // one strategy
orders.sort(Comparator.comparing(Order::date).reversed()); // another
```

Builder, for readable construction

```java
var request = HttpRequest.newBuilder()
    .uri(URI.create(url))
    .header("Accept", "application/json")
    .timeout(Duration.ofSeconds(10))
    .GET()
    .build();   // fluent, readable, immutable result
```

Factory and Singleton, modernised

- **Factory** — static factory methods (`List.of`, `Optional.of`) over constructors.
- **Singleton** — a single-element `enum` is the simplest, safest form.
- **Value objects** — `record`s replace hand-written immutable classes.
- **State machines** — sealed types plus pattern matching model states cleanly.

Patterns as vocabulary

Patterns remain valuable as shared vocabulary — saying ‘use an Observer’ or ‘this is a Decorator’ conveys design intent instantly. Modern Java changes the implementation, not the ideas.

© 2026 ByteMart Digital

TIP

Do not force patterns onto simple problems. The best modern-Java solution is often a lambda, a record, or a sealed type — reach for a full pattern only when the extra structure genuinely earns its keep.

Same ideas, lighter code

Design patterns still solve real problems, but modern Java expresses many with a fraction of the ceremony. Learn the patterns as vocabulary, then implement them the modern way — with lambdas, records, and sealed types.